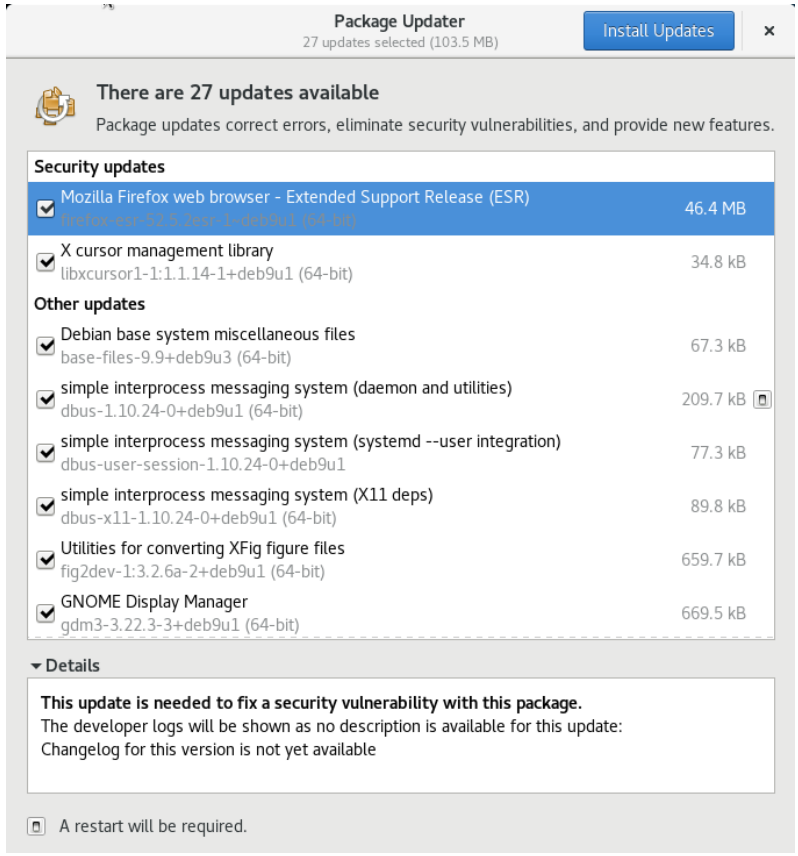


Figure 6.3 *Upgrading with `gpk-update-viewer`*

This tool is no longer installed in the default GNOME desktop. The new philosophy is that security updates should be automatically installed, either in the background or, preferably, when you shutdown your computer so as to not confuse any running application.

6.9. Automatic Upgrades

Since Falcot Corp has many computers but only limited manpower, its administrators try to make upgrades as automatic as possible. The programs in charge of these processes must therefore run with no human intervention.

6.9.1. Configuring `dpkg`

As we have already mentioned (see sidebar "Avoiding the configuration file questions" page 89), `dpkg` can be instructed not to ask for confirmation when replacing a configuration file (with the `--force-confdef --force-confold` options). Interactions can, however, have three other sources:

some come from APT itself, some are handled by `debconf`, and some happen on the command line due to package configuration scripts (sometimes handled by *ucf*).

6.9.2. Configuring APT

The case of APT is simple: the -y option (or --assume-yes) tells APT to consider the answer to all its questions to be "yes".

6.9.3. Configuring `debconf`

The case of `debconf` deserves more details. This program was, from its inception, designed to control the relevance and volume of questions displayed to the user, as well as the way they are shown. That is why its configuration requests a minimal priority for questions; only questions above the minimal priority are displayed. `debconf` assumes the default answer (defined by the package maintainer) for questions which it decided to skip.

The other relevant configuration element is the interface used by the front-end. If you choose `noninteractive` out of the choices, all user interaction is disabled. If a package tries to display an informative note, it will be sent to the administrator by email.

To reconfigure `debconf`, use the `dpkg-reconfigure` tool from the *debconf* package; the relevant command is `dpkg-reconfigure debconf`. Note that the configured values can be temporarily overridden with environment variables when needed (for instance, `DEBIAN_FRONTEND` controls the interface, as documented in the `debconf(7)` manual page).

6.9.4. Handling Command Line Interactions

The last source of interactions, and the hardest to get rid of, is the configuration scripts run by `dpkg`. There is unfortunately no standard solution, and no answer is overwhelmingly better than another.

The common approach is to suppress the standard input by redirecting the empty content of `/dev/null` into it with *`command`* `</dev/null`, or to feed it with an endless stream of newlines. None of these methods is 100 % reliable, but they generally lead to the default answers being used, since most scripts consider a lack of reply as an acceptance of the default value.

6.9.5. The Miracle Combination

By combining the previous elements, it is possible to design a small but rather reliable script which can handle automatic upgrades.

Example 6.5 *Non-interactive upgrade script*

```
export DEBIAN_FRONTEND=noninteractive
yes '' | apt-get -y -o DPkg::options::="--force-confdef" -o DPkg::options::="--force-
    ➥ confold" dist-upgrade
```

IN PRACTICE

The Falcot Corp case

Falcot computers are a heterogeneous system, with machines having various functions. Administrators will therefore pick the most relevant solution for each computer.

In practice, the servers running *Buster* are configured with the "miracle combination" above, and are kept up to date automatically. Only the most critical servers (the firewalls, for instances) are set up with `apticron`, so that upgrades always happen under the supervision of an administrator.

The office workstations in the administrative services also run *Buster*, but they are equipped with *gnome-packagekit*, so that users trigger the upgrades themselves. The rationale for this decision is that if upgrades happen without an explicit action, the behavior of the computer might change unexpectedly, which could cause confusion for the main users.

In the lab, the few computers using *Testing* — to take advantage of the latest software versions — are not upgraded automatically either. Administrators only configure APT to prepare the upgrades but not enact them; when they decide to upgrade (manually), the tedious parts of refreshing package lists and downloading packages will be avoided, and administrators can focus on the really useful part.

6.10. Searching for Packages

With the large and ever-growing amount of software in Debian, there emerges a paradox: Debian usually has a tool for most tasks, but that tool can be very difficult to find amongst the myriad other packages. The lack of appropriate ways to search for (and to find) the right tool has long been a problem. Fortunately, this problem has almost entirely been solved.

The most trivial search possible is looking up an exact package name. If `apt show` *`package`* returns a result, then the package exists. Unfortunately, this requires knowing or even guessing the package name, which isn't always possible.

TIP

Package naming conventions

Some categories of packages are named according to a conventional naming scheme; knowing the scheme can sometimes allow you to guess exact package names. For instance, for Perl modules, the convention says that a module called `XML::Handler::Composer` upstream should be packaged as *libxml-handler-composer-perl*. The library enabling the use of the `gconf` system from Python is packaged as *python-gconf*. It is unfortunately not possible to define a fully general naming scheme for all packages, even though package maintainers usually try to follow the choice of the upstream developers.

A slightly more successful searching pattern is a plain-text search in package names, but it remains very limited. You can generally find results by searching package descriptions: since

each package has a more or less detailed description in addition to its package name, a keyword search in these descriptions will often be useful. `apt-cache` and `axi-cache` are the tools of choice for this kind of search (see "`axi-cache`" page 126); for instance, `apt-cache search video` will return a list of all packages whose name or description contains the keyword "video".

For more complex searches, a more powerful tool such as `aptitude` is required. `aptitude` allows you to search according to a logical expression based on the package's meta-data fields. For instance, the following command searches for packages whose name contains kino, whose description contains video and whose maintainer's name contains paul:

```
$ aptitude search kino~dvideo~mpaul
p   kino  - Non-linear editor for Digital Video data
$ aptitude show kino
Package: kino
Version: 1.3.4+dfsg0-1
State: not installed
Priority: optional
Section: video
Maintainer: Paul Brossier <piem@debian.org>
Architecture: amd64
Uncompressed Size: 8,304 k
Depends: libasound2 (>= 1.0.16), libatk1.0-0 (>= 1.12.4), libavc1394-0 (>= 0.5.3),
    ➥ libavcodec58 (>=
        7:4.0) | libavcodec-extra58 (>= 7:4.0), libavformat58 (>= 7:4.0),
            ➥ libavutil56 (>= 7:4.0),
        libc6 (>= 2.14), libcairo2 (>= 1.2.4), libdv4 (>= 1.0.0), libfontconfig1 (>=
            ➥  2.12.6),
        libfreetype6 (>= 2.2.1), libgcc1 (>= 1:3.0), libgdk-pixbuf2.0-0 (>= 2.22.0),
            ➥  libglade2-0
        (>= 1:2.6.4-2~), libglib2.0-0 (>= 2.16.0), libgtk2.0-0 (>= 2.24.32), libice6
            ➥  (>= 1:1.0.0),
        libiec61883-0 (>= 1.2.0), libpango-1.0-0 (>= 1.14.0), libpangocairo-1.0-0
            ➥ (>= 1.14.0),
        libpangoft2-1.0-0 (>= 1.14.0), libquicktime2 (>= 2:1.2.2), libraw1394-11,
            ➥ libsamplerate0
        (>= 0.1.7), libsm6, libstdc++6 (>= 5.2), libswscale5 (>= 7:4.0), libx11-6,
            ➥ libxext6,
        libxml2 (>= 2.7.4), libxv1, zlib1g (>= 1:1.1.4)
Recommends: ffmpeg, curl
Suggests: udev | hotplug, vorbis-tools, sox, mjpegtools, lame, ffmpeg2theora
Conflicts: kino-dvtitler, kino-timfx, kinoplus
Replaces: kino-dvtitler, kino-timfx, kinoplus
Provides: kino-dvtitler, kino-timfx, kinoplus
Description: Non-linear editor for Digital Video data
 Kino allows you to record, create, edit, and play movies recorded with DV camcorders
     ➥ . This program
 uses many keyboard commands for fast navigating and editing inside the movie.
```

```
 The kino-timfx, kino-dvtitler and kinoplus sets of plugins, formerly distributed as
     ➥ separate
 packages, are now provided with Kino.
Homepage: http://www.kinodv.org/
Tags: field::arts, hardware::camera, implemented-in::c, implemented-in::c++,
    ➥ interface::graphical,
      interface::x11, role::program, scope::application, suite::gnome, uitoolkit::gtk
          ➥ ,
      use::editing, use::learning, works-with::video, x11::application
```

The search only returns one package, *kino*, which satisfies all three criteria.

Even these multi-criteria searches are rather unwieldy, which explains why they are not used as much as they could. A new tagging system has therefore been developed, and it provides a new approach to searching. Packages are given tags that provide a thematical classification along several strands, known as a "facet-based classification". In the case of *kino* above, the package's tags indicate that Kino is a Gnome-based software that works on video data and whose main purpose is editing.

Browsing this classification can help you to search for a package which corresponds to known needs; even if it returns a (moderate) number of hits, the rest of the search can be done manually. To do that, you can use the ~G search pattern in `aptitude`, but it is probably easier to simply navigate the site where tags are managed:

➨ `https://debtags.debian.org/`

Selecting the works-with::video and use::editing tags yields a handful of packages, including the *kino* and *pitivi* video editors. This system of classification is bound to be used more and more as time goes on, and package managers will gradually provide efficient search interfaces based on it.

To sum up, the best tool for the job depends on the complexity of the search that you wish to do:

- `apt-cache` only allows searching in package names and descriptions, which is very convenient when looking for a particular package that matches a few target keywords;
- when the search criteria also include relationships between packages or other meta-data such as the name of the maintainer, `synaptic` will be more useful;
- when a tag-based search is needed, a good tool is `packagesearch`, a graphical interface dedicated to searching available packages along several criteria (including the names of the files that they contain). For usage on the command-line, `axi-cache` will fit the bill.
- finally, when the searches involve complex expressions with logic operations, the tool of choice will be `aptitude`'s search pattern syntax, which is quite powerful despite being somewhat obscure; it works in both the command-line and the interactive modes.

